

Programación Web
Unidad 12: Despliegue y CI/CD
Post-Contenido 2 — Proyecto:
Pipeline CI/CD con GitHub Actions y Docker Hub

Abrahan Remolina Bolivar - 02230131004

Docente:
Jonathan Ronaldo Rey Castillo

Universidad De Santander
UDES - Campus Cúcuta
Cucuta, Norte De Santander
2026

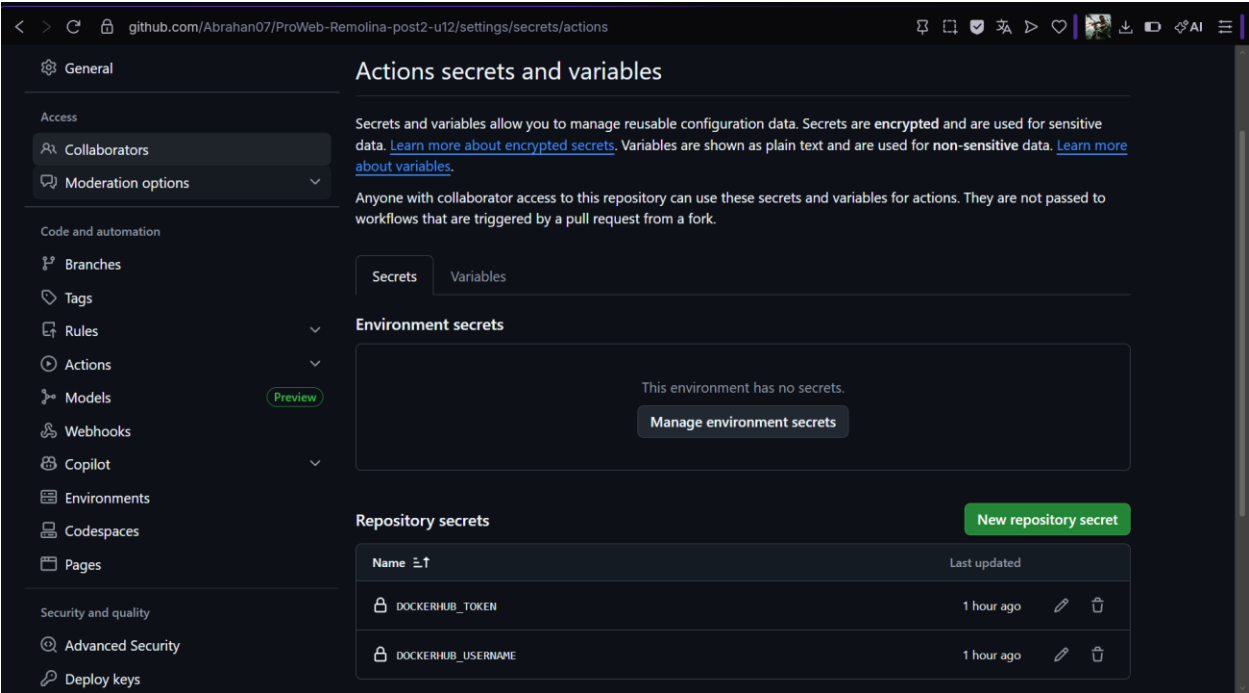
1. INTRODUCCIÓN

En este proyecto se diseñó e implementó un pipeline de integración y entrega continua (CI/CD) utilizando GitHub Actions y Docker Hub. El pipeline automatiza la compilación del proyecto Spring Boot, la ejecución de pruebas unitarias con reporte de cobertura JaCoCo, la construcción de la imagen Docker con multi-stage build y su publicación en Docker Hub, activándose automáticamente en cada push a la rama principal del repositorio.

2. CONFIGURACIÓN DE GITHUB SECRETS

Para autenticar el pipeline con Docker Hub se configuraron dos secrets en el repositorio bajo Settings → Secrets and variables → Actions:

- **DOCKERHUB_USERNAME:** nombre de usuario de Docker Hub (abraham07)
- **DOCKERHUB_TOKEN:** Access Token generado en Docker Hub con permisos de lectura y escritura

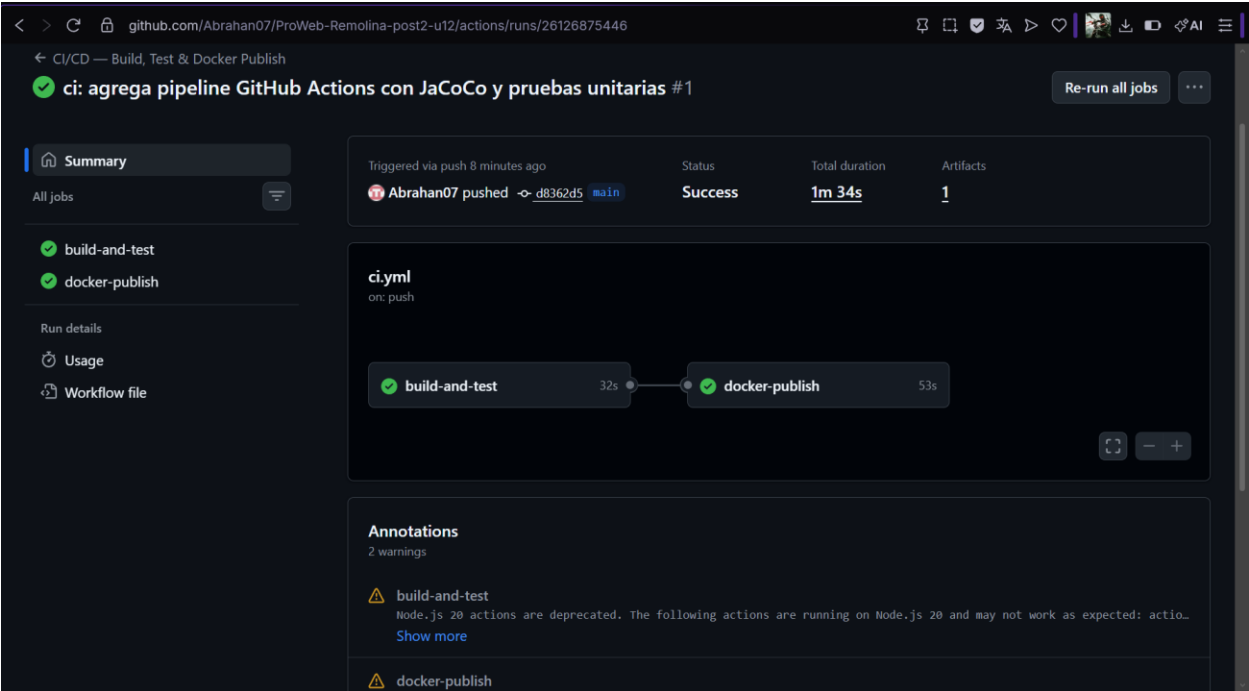


3. PIPELINE DE GITHUB ACTIONS

El archivo `.github/workflows/ci.yml` define dos jobs encadenados:

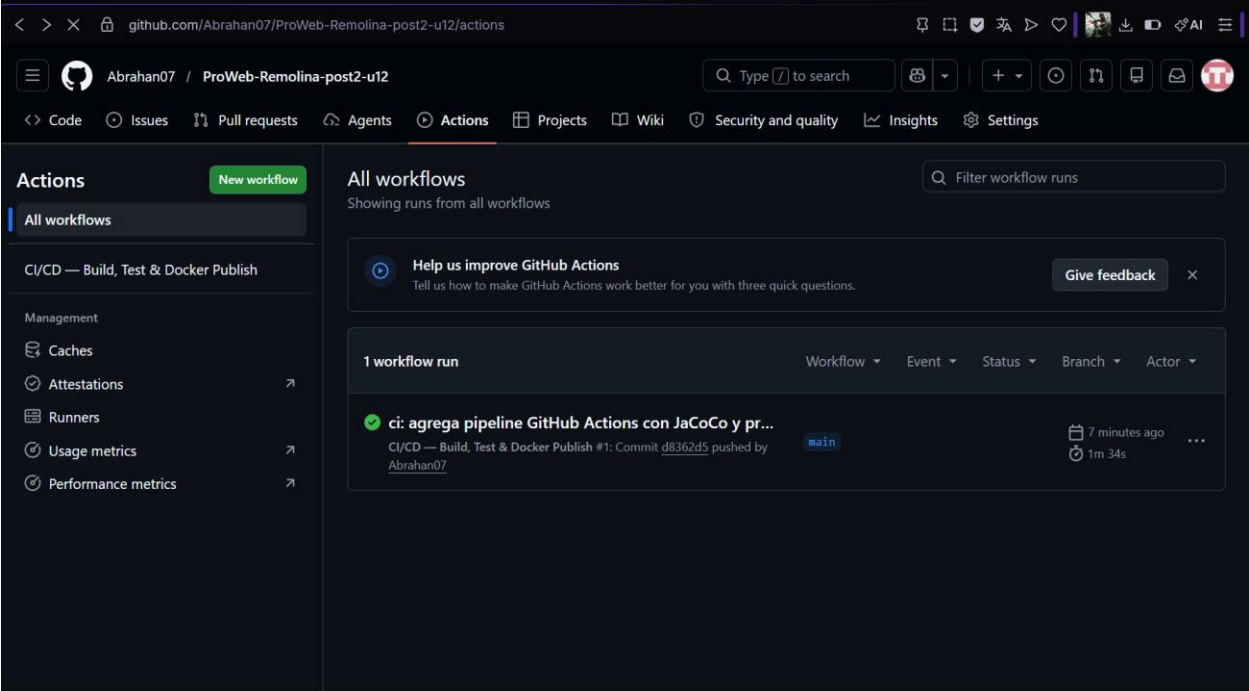
Job 1 — build-and-test: configura JDK 21 con caché Maven, compila el proyecto con `mvn clean verify`, ejecuta las pruebas unitarias y publica el reporte JaCoCo como artefacto descargable con retención de 7 días.

Job 2 — docker-publish: se ejecuta únicamente en push a main (no en pull requests), depende del job anterior mediante `needs: build-and-test`, se autentica en Docker Hub con los secrets configurados, construye la imagen con multi-stage build y la publica con dos tags: `latest` y `sha-<commit>`.



4. HISTORIAL DE EJECUCIONES EN GITHUB ACTIONS

El historial de la pestaña Actions del repositorio muestra las ejecuciones del workflow con check verde, confirmando que el pipeline se ejecuta correctamente en cada push a main.



5. REPORTE DE COBERTURA JACOCO

El job build-and-test genera automáticamente el reporte de cobertura JaCoCo y lo publica como artefacto descargable dentro del workflow run. El reporte muestra la cobertura de las clases ProductoService y demás componentes probados con Mockito.

index.html

Archivo C:/Users/abr/AppData/Local/Temp/24d8f995-b037-4fad-91e8-483ee6067141_jacoco-report%20(1).zip.141/index.html

Chat

Sessions

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.universidad.remolina_post1_u12	54	0%	2	0%	5	5	11	11
com.universidad.remolina_post1_u12.controller	33	0%		n/a	5	5	10	5
com.universidad.remolina_post1_u12.model	24	15		n/a	7	9	7	13
com.universidad.remolina_post1_u12.service	10	16		n/a	2	5	3	8
Total	121 of 152	20%	2 of 2	0%	19	24	31	42

Created with JaCoCo 0.8.11.202310140853

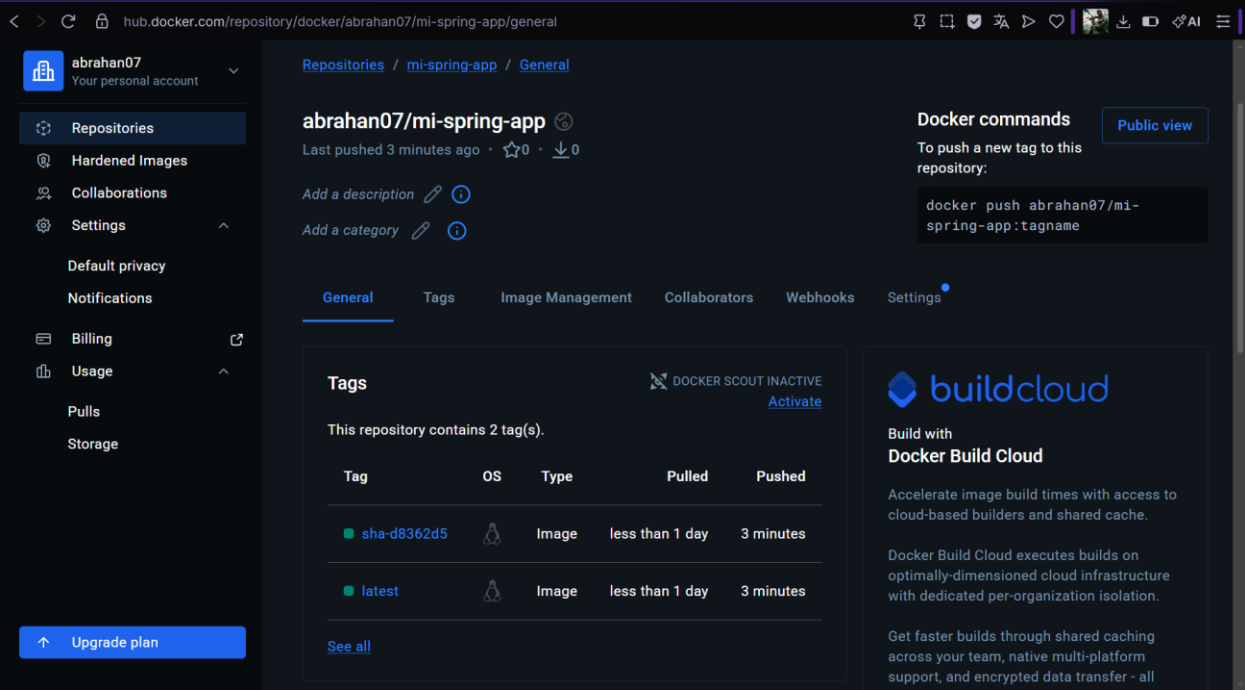
6. IMAGEN PUBLICADA EN DOCKER HUB

La imagen fue publicada exitosamente en Docker Hub bajo el repositorio abrahan07/mi-spring-app con los siguientes tags:

- latest — imagen de la última versión en main
- sha-d8362d5 — tag con el SHA del commit que disparó el pipeline

La imagen puede descargarse con:

```
docker pull abrahan07/mi-spring-app:latest
```



7. CONCLUSIONES

Se implementó exitosamente un pipeline CI/CD con GitHub Actions que automatiza todo el flujo desde la compilación hasta la publicación de la imagen en Docker Hub. El uso de GitHub Secrets garantiza que las credenciales nunca quedan expuestas en el código. El encadenamiento de jobs con needs : asegura que la imagen solo se publica si las pruebas pasan correctamente, lo que representa una buena práctica de integración continua.